

Blinded-V5: the LGI-based compute module

design rationale, construction, parameters, and measurements

local-mixing project

2026-09-04

Where it fits in the pipeline

A secret circuit C on n wires is protected in two steps: it is first **gadgetized** into an equivalent but structurally-scrambled circuit on $2n$ wires, which is then **mixed** (fmix). Blinded-V5 lives inside the gadgetizer, and this document assumes you know the earlier gadgetize designs but nothing about V5.

The **gadgetize module** is a **5-stage** pipeline, each stage on $2n$ wires (data $0..n$, band/junk $n..2n$):

1. **slice** — a junk-guard zero-slice keyed on the band (dead at the input port, where the band is 0);
2. **seed the band wires** — each band wire set to $x_i \wedge \neg x_j$ from the honest inputs;
3. **compute** — realise the input circuit A on the $2n$ wires so the data half carries A 's output and the band is only read;
4. **re-seed / re-randomise the band wires** — band turnover;
5. **re-slice** — the closing junk-guard slice (fires at the output port).

One term to keep straight: the circuit A that the gadgetizer computes is **not** C **itself**. It is the **sliced sandwich of C** — the output of a *separate sandwich module* that wraps C (with a random D , interleaved slice blocks, a floated column) into a $2n$ -wire circuit with $A(x, 0) = (\text{junk}, C(x))$. The sandwich *generates* the circuit that the gadgetize module then computes and mixes; it is a different module and is not one of the five stages above.

Blinded-V5 is a drop-in replacement for stage 3 — the compute. Where the earlier compute stages (the drip **route_fire**, the product-share carriers) realise A by *routing* each operand into position and firing a gadget there, Blinded-V5 instead embeds A **inside one enormous masking identity**: the compute's body is a cloud of long masking identities on the band, and A 's gates fire from *within* the mask by a momentary, reversible unmasking of their operands. It takes A and emits an equivalent A' on the $2n$ wires; the other four stages are unchanged. (V5 re-randomises the band *internally* as part of its own masking hygiene — §2 — which is distinct from the separate stage-4 re-seed.)

Source: `src/preprocessing/blinded.v5.rs`. Driver: `gen_sandwich_gadget ... blinded-v5;`
pipeline: `gss_mix.sh --gadgetization-mode blinded-v5`. The Markdown twin of this document is `BLINDED_V5_LGI_DESIGN.md`.

1 Why this design — the rationale

The compute stage must move the intermediate state as far as possible from the plaintext computation A while keeping A' exactly equivalent to A on the data wires. The design is a chain of forced moves.

(a) **Long identities are the only thing that moves the state.** Across the HD and affine heatmaps, short local edits barely displace the state; the ridge of recoverable structure only recedes when the masking identity woven through a wire is *long* — comparable to the circuit itself.

(b) **The only long identity that moves the state is the commutative structure of an identity with a single active wire.** A run of gates all targeting one active wire w and reading only controls is an identity iff the increments XOR to zero, and *all its gates commute* — which lets it be arbitrarily long and freely interleaved.

(c) **Entangle identities on different active wires by sharing the control wires — the band.** Every identity targets a data wire and reads only the band ($n..2n$), so identities on different active wires also commute; a band wire participates in the masks of many data wires at once.

(d) **Add random updates of the control wires from the active wires — in bursts.** Band updates **re-randomise** the controls (fresh band entropy throughout the run, not just at the seed) and **break naive commutation-back** (once a band wire is updated mid-run, the identities reading it before and after no longer commute past it), without ruining correctness (each straddling identity is closed before the update or re-derived across it). The updates are emitted in **bursts**: one *slot* is $F \approx 8K$ gates all targeting a single band wire b , each control drawn independently as a live-data or a band wire (so a burst spans $b \oplus= data \wedge aux$, $b \oplus= aux \wedge aux$, and $b \oplus= data \wedge data$). Bursting **concentrates** the band mixing into few slots (so fewer straddle-closures interfere with the LGIS) and gives each band wire a **data-wire-like burst-of-activity signature**.

(e) **Embed A inside one circuit-spanning identity via hidden unmasking.** Each gate fires from inside the mask: its operands are momentarily and reversibly unmasked into a linear combination of band wires (never bare), it fires as an expansion over those masked wires, then re-masks.

That is the whole design: a long, entangled, self-updating masking cloud with A firing from inside it. Everything in §2 is *how* to realise (a)–(e) as a single correct circuit.

2 The construction

A has n wires and m gates; A' has $2n$. Let u_w be the uses of wire w in A and w_w its writes.

Masking atom. $\text{g57}(w, x, y) = w \oplus= 1 \oplus (\neg x \wedge y)$ (data target, band controls). A disjoint-pair LGI $\bigoplus_i \text{g57}(w, cy_{2i}, cy_{2i+1})$ is a deg-2 mask; a **g57** and its reverse **linearise**, $\text{g57}(w, r_1, r_2) \oplus \text{g57}(w, r_2, r_1) = w \oplus r_1 \oplus r_2$ (AND monomials do not). Disjoint pairs, not a K -cycle (which would telescope to a bare operand); $K \geq 2$.

Why one pass — co-sampling. The masks, the band updates, and A ’s gates are **co-sampled**: produced together in a single forward pass, rather than laying a fixed masking scaffold first and threading A through it afterwards. The reason is hidden firing (§3): every A -gate must be straddled by a *real*, rerand-protected masking identity opened at the exact instant the gate fires. Only a co-sampled pass can open that identity on demand, so the firing-hiding is complete while reusing the wire’s existing mask budget — at no size cost. (A fixed scaffold, filled in afterwards, starves at the tail and covers only 60–70% of gates; §3.)

The algorithm. A cheap setup, then a forward pass.

Setup.

1. **Mask budget.** Give each active wire w exactly $u_w + 1$ masking identities, split into w_w **straddle opens** (one per write to w , opened *on demand* at the gate that writes it — for hidden firing) and $\text{reads}_w + 1$ **filler opens** (opened between gates to keep w masked while it is read). At most **max_open** are open on a wire at once. This is the same budget a fixed scaffold would use, so the statistics are unchanged.
2. **Order.** Build the data-hazard DAG of A (**compute_deps**): a read is ordered after every earlier write of the wire, and a write after every earlier read — but same-target XOR writes commute, so there is *no* write-after-write edge. Any linear extension preserves A ’s function; a Kahn **ready queue** holds the gates whose predecessors are all placed.
3. **Rerand plan.** A shuffled list of **straddle_slots** + **repair_slots** band-update *slots* (§1d; auto **straddle_slots** $\approx m/(4K)$, **repair_slots** = 0).
4. **Rate.** Fire one rerand slot every $\text{total_steps}/\#\text{slots}$ primitive steps (a *step* = one A -gate placement or one filler open; $\text{total_steps} = m + \Sigma \text{filler}$), so the slots spread evenly and the last lands *during* the pass — no end-flush (a surplus of requested slots over total_steps warns rather than truncating silently).

Forward pass — repeat until all m gates are placed:

1. **Pop** a ready A -gate g (target wire c).
2. **Masked read.** For each operand wire, *linearise* its currently-open masks — complete each open **g57** with its reverse so the wire carries $\text{operand} \oplus \rho$, ρ a linear XOR of band wires — topping up with fresh disjoint pairs until $|\rho| \geq \text{min_mask}$ (a **hard floor**: never bare, and never thinly masked even when the open pairs happen to cancel). Remember the reverse gates as the *undo*.
3. **Fire from inside the mask.** Expand $c \oplus = \text{comp} \oplus \text{lit}(a) \wedge \text{lit}(b)$ as $(a' \oplus \rho_a)(b' \oplus \rho_b)$ into a batch of monomials over **ONLY** the masked control wires and band wires — never bare a , b , or $a \oplus b$ (0/1/2 controls and all polarities; the 0-control fire is $\neg \text{comp}$).
4. **Straddle (hidden firing, §3).** Split the monomial batch into two halves, shuffle each **independently** (they all target c and commute), emit the first half, **open a fresh lgi on c here** (one of its straddle opens, generated on demand), then emit the second half. The mid-fire open makes the module’s net XOR on c equal $\Delta \oplus \text{secret-mask}$, not the bare increment Δ .
5. **De-linearise** — emit the undo gates, restoring the operands’ masks.

6. **Maybe rerand** — if the calibrated rate says so, emit one band-update slot (below).

7. **Release & fill** — decrement the in-degree of g 's dependents, moving any that reach zero into the ready queue; then open a few **filler** LGIs on wires chosen by remaining filler budget.

Finish. Drain any remaining filler opens, then close every still-open LGI so the data wires end holding A 's true output.

The rerand slot (step 6) is a §1d burst: one band wire b , $F \approx 8K$ updates. A STRADDLE slot first **closes** the masks reading b (this is what thins masking past the ≈ 1024 knee — hence keeping **straddle_slots** at that budget); a REPAIR slot brackets the burst with each such mask's b -reading **g57** (old b cancels, new b re-adds, so the mask stays open — no thinning). Running in the same pass, the band updates protect the straddle opens automatically.

Correctness is verified exhaustively over all 2^n inputs $\times$ many band settings for $n \leq 6$, $K \in 2..5$, all **max_open** and both rerand kinds (**scratchpad/v6**: 891 gadgets, 0 mismatches, plus a 360-case reorder-preserves- A check), and end-to-end in **gen_sandwich_gadget** (forward + reverse-honesty sample-verify PASS).

3 Hidden firing

An *atomic*, mask-restoring gate module would leak which gate fired: the active wire's before/after XOR across it equals the true increment $\Delta = comp \oplus lit(a) \wedge lit(b)$. That per-module increment is a robust invariant that survives the downstream mixing, so an adversary who segments the mixed circuit at module boundaries reads off A 's gate list.

The fix uses the abundant masking gates on the *same* active wire: split the fire and emit one of c 's LGI **opens between the halves**. It toggles c mid-fire and stays open past the module, so the net XOR on c is $\Delta \oplus (\text{secret band-mask})$ — never the bare increment. Because everything is co-sampled, the straddling LGI is a real, rerand-protected scaffold LGI opened exactly when the gate is placed, so **every** gate is covered, reusing the wire's $u_w + 1$ budget at **no size cost** (measured: 7920/7920 gates on the $n=128$ sandwich). Earlier attempts — A -gate weaving ($\sim 23\%$), an injected firing mask (100% but $+43\%$ size), straddling fixed-scaffold slots (60–70%, tail slot-starved) — are superseded by generating the slot on demand.

4 Parameters and why

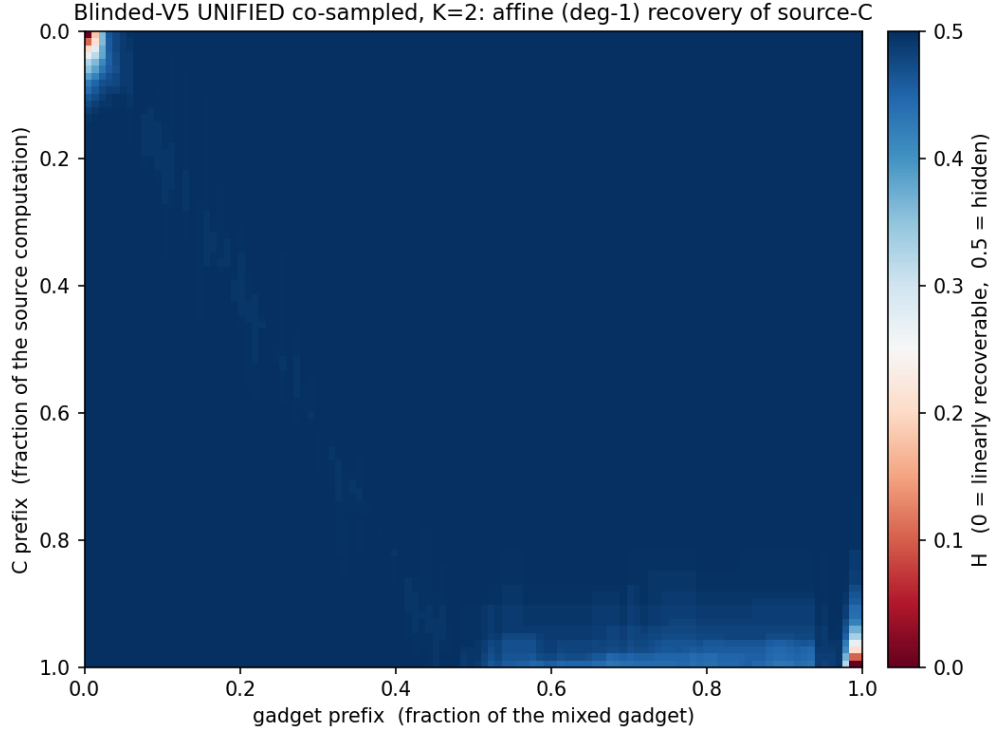
knob	prod.	meaning	why this value
K	2	band wires per LGI = per-LGI mask width ($\lfloor K/2 \rfloor$ pairs; $ \rho \approx \text{max_open} \cdot K$) — not the temporal length	affine/deg-2-neutral across K (§5); smallest is best. Size $\sim$ linear, read cost quadratic in $ \rho $, so large K explodes. Odd K wastes a wire.
<code>max_open</code>	3	rolling cap on simultaneously-open LGIs per wire	wider ρ = more local hiding, but read cost is quadratic in $ \rho $; 3 is the knee.
<code>min_mask</code>	auto = <code>max_open</code> = 3	hard floor on $ \rho $ (masking wires) per operand read	guarantees no operand is ever read under fewer than 3 masking wires, even in a rare draw where the open pairs cancel (worst read $ \rho $ rises $2 \rightarrow 4$; mean $ \rho \approx 5.9$ unchanged; +0.2% gates).
<code>rerand_level</code>	auto = $m/(4K)$ (≈ 875)	close-straddling-masks band-update slots	at the safe side of the ≈ 1024 thinning knee; the <i>slot</i> count (not the gate count) is what thins.
<code>rerand_repair</code>	0	re-derive-across-update band-update slots (no thinning)	off by default; add slots for extra band turnover at no masking cost.
<code>rerand_burst</code> (F)	auto = 8K (=16 at $K=2$)	band-update gates per slot (the burst)	comparable to a data wire’s write-burst (tens of gates), so band wires carry a data-wire-like activity signature; $\text{slots} \times F \approx 2m$.

K sets each LGI’s mask *width*, not its temporal length — the identity’s circuit-spanning length is a property of the whole scaffold (§1e).

5 Measurements

All measurements are on the current co-sampled build with **burst rerand + the masking floor** (n=128, $|C| = |D| = 3000$, $K=2$). They match the pre-firing-fix baseline to within noise — neither the hidden-firing redesign nor the burst/floor refinements moved them (the burst change touches only the band wires, so it cannot move the data-wire ridge): degree-1 meanH = 0.4966, degree-2 meanH = 0.4967, exposed- $C < 0.35 \approx 5.6$ –5.7%.

5.1 Affine ridge

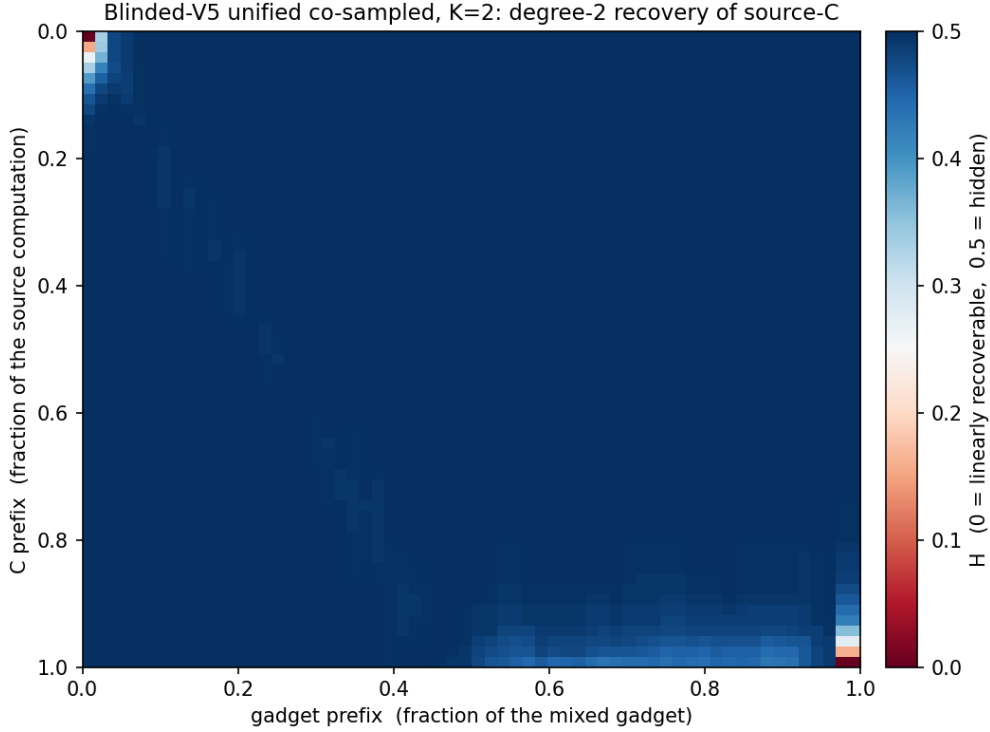


mean $H \approx 0.498$, ridge depth ≈ 0.05 ; the interior ($\sim 78\%$ of rows) is fully hidden ($H \approx 0.49$), and the only recovery is a thin fringe at the two endpoints.

5.2 The exposed fringe is public I/O — input segments are not leaked

The $\approx 6\text{--}10\%$ of source- C segments that are linearly exposed ($\text{min-}H < 0.25\text{--}0.35$) are the two corners of the diagonal. The **input side is masked as soon as the compute begins** — recovery is confined to the extreme top-left corner, i.e. the raw input port where x is public *before* any masking. The output side is the necessarily-delivered $C(x)$. So the input wire segments show a **lack of linear exposure** beyond the trivial public input; the boundary-cushion knobs measured no effect because the fringe is unavoidable public I/O, not a leak.

5.3 Degree-2 recovery



meanH ≈ 0.497 , depth ≈ 0.05 . The quadratic adversary reaches only ~ 0.01 deeper than affine, into the same low-degree I/O fringe; the interior stays fully hidden. Degree-2 is flat across K , like affine.

5.4 Rerand: bursts, straddle vs repair ($n_A = 256$, affine deg-1)

Each rerand slot is a **burst** of $F \approx 8K$ gates on one band wire; the **slot** count — not the gate count — drives the effect, since a straddle slot closes the masks reading b exactly once per slot. An early single-gate-dose study fixes where the thinning knee is (dose = total straddle/repair *gates*):

straddle	repair	gadget	meanH	affDepth
0	0	670k	0.4979	0.0550
1000	3000	668k	0.4959	0.0514
8000	0	607k	0.4878	0.0681
0	8000	780k	0.4985	0.0549

Heavy STRADDLE thins (meanH $\downarrow$, ridge $\uparrow$, gadget *shrinks* as masks close); heavy REPAIR does not (both = baseline, gadget *grows* from the pre/post pairs). The production plan keeps straddle **slots** on the safe side of the ≈ 1024 knee: auto $\approx m/(4K)$ (~ 875 for the half-size sandwich) $\times F = 8K \approx 2m$ band-refresh gates, **no repair**. Fewer, fatter slots mix the band as hard as many single updates while interfering less with the LGIs (and give the band its data-wire signature), and — because the thinning is driven by the *slot* count — they sit at the baseline floor (deg-1 meanH = 0.4966, deg-2 0.4967).

5.5 Through the GSS pipeline ($K = 2$ vs $K = 4$)

Pending — from the co-sampled `gss_mix.sh --gadgetization-mode blinded-v5` runs ($n=128$, half-size $|C| = |D| = 3000$; grow-to-2 $\times$ then hold ≈ 10 effs, 2-eff snapshot; then split, crossing, fcompress), $K = 2$ and $K = 4$: affine ridge and exposed- C count at the 2-eff snapshot and at the end, with stage sizes.

6 Tradeoffs and the current choices

- **Mask width K vs size.** Larger K widens each mask, not the temporal length; hiding is flat in K while size grows $\sim$ linearly and the read cost quadratically, so **K=2**; K=4 only for a wider read-mask.
- **Band turnover vs size (bursts, slots vs F).** More band updates = more entropy and stronger resistance to commutation-back, but STRADDLE **slots** thin the masking past ≈ 1024 and REPAIR slots grow the gadget. Bursting decouples the two knobs: F (gates/slot) sets how hard each band wire is stirred and how data-like its activity looks, while the **slot count** ($\approx m/4K$, held under the knee) sets the thinning. Auto $m/4K$ slots $\times F = 8K$, no repair, sits at the baseline floor.
- **Masking floor is a cheap hedge.** `min_mask` guarantees every operand is read under ≥ 3 masking wires (worst-case $|\rho|$ $2 \rightarrow 4$; mean unchanged); it fires only on the rare under-floor draw, costing $\sim 0.2\%$ gates — a hedge against a low-probability thin-masking event, not a routine cost.
- **Firing-hiding is free** in the co-sampled build (it reuses the existing LGI budget); earlier fixed-scaffold approaches paid coverage (23–70%) or +43% size.
- **The residual fringe is I/O, not a knob** — no parameter removes the public input/output boundary.